Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Операционные системы

Лабораторная работа №2

Выполнил: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2023

Цель: знакомство с инструментарием для разработки программ на платформе Linux.

Задание:

1. Написать программу для манипуляции данными на основе рекурсивных структур, реализовать функции вставки, удаления и навигации для списка; реализовать сериализацию списка. (Например: электронный журнал успеваемости студентов.)
2. Провести трассировку программы с помощью gdb.

Инструментарий: редактор vim, компилятор gcc, отладчик gdb. Язык C.

Ход работы:

1. Создание файла. Вход в текстовый редактор для написания кода:

```
nothomepc@DESKTOP-JB25TSP:~$ touch 1lab.c
nothomepc@DESKTOP-JB25TSP:~$ vim 1lab.c
```

2. Выход из vim после написания программы:

nothomepc@DESKTOP-JB25T

```
        return;
    }

    struct Student* current = head;
    while (current != NULL) {
        fwrite(current, sizeof(struct Student), 1, file);
        current = current->next;
    }

    fclose(file);
    printf("Сериализация завершена.\n");
}

int main() {
    insertStudent("Иван", 101, 85.5);
    insertStudent("Мария", 102, 92.0);
    insertStudent("Петр", 103, 78.2);

    displayStudents();

    deleteStudent(102);

    displayStudents();

    serializeStudents("journal.dat");

    return 0;
}

:wq
```

3. Компиляция, запуск программы и вывод результатов работы:

```
nothomepc@DESKTOP-JB25TSP:~$ gcc -o app -Wall 1lab.c
nothomepc@DESKTOP-JB25TSP:~$ ls
1lab.c  app  code  jscl
nothomepc@DESKTOP-JB25TSP:~$ ./app
Студент добавлен в журнал.
Студент добавлен в журнал.
Студент добавлен в журнал.
Журнал успеваемости студентов:
Имя: Петр, Номер: 103, Оценка: 78.20
Имя: Мария, Номер: 102, Оценка: 92.00
Имя: Иван, Номер: 101, Оценка: 85.50
Студент удален из журнала.
Журнал успеваемости студентов:
Имя: Петр, Номер: 103, Оценка: 78.20
Имя: Иван, Номер: 101, Оценка: 85.50
Сериализация завершена.
nothomepc@DESKTOP-JB25TSP:~$ ls
1lab.c  app  code  journal.dat  jscl
nothomepc@DESKTOP-JB25TSP:~$
```

4. Компиляция программы для отладки и запуск отладчика:

```
nothomepc@DESKTOP-JB25TSP:~$ gcc -g -o app -Wall 1lab.c
nothomepc@DESKTOP-JB25TSP:~$ gdb ./app
```

5. Основные команды в gdb:
   - **break** (или **b**): устанавливает точку останова. Например, **break main (b main)** устанавливает точку останова в функции **main**.
   - **step** (или **s**): выполняет текущую инструкцию и переходит внутрь функции, если таковая есть.
   - **next** (или **n**): выполняет текущую инструкцию и переходит к следующей строке кода в текущей функции.
   - **print** (или **p**): позволяет просматривать значения переменных. Например, **print x** отобразит значение переменной **x**.
   - **quit** (или **q**): Завершает GDB.
6. Отладка программы в gdb:

```
(gdb) b main
Breakpoint 1 at 0x14ad: file 1lab.c, line 83.
(gdb) r
Starting program: /home/nothomepc/app
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at 1lab.c:83
83          insertStudent("Иван", 101, 85.5);
(gdb) s
insertStudent (name=0x5555555561cd "Иван", roll_number=101, grade=85.5) at 1lab.c:18
18          struct Student* newStudent = (struct Student*)malloc(sizeof(struct Student));
(gdb) n
19          strncpy(newStudent->name, name, sizeof(newStudent->name));
```

```
39                  free(current);
(gdb) n
40                  printf("Студент удален из журнала.\n");
(gdb) print current
$2 = (struct Student *) 0x555555559700
(gdb) n
Студент удален из журнала.
41                  return;
(gdb) n
48      }
(gdb) n
main () at 1lab.c:91
91          displayStudents();
(gdb) n
Журнал успеваемости студентов:
Имя: Петр, Номер: 103, Оценка: 78.20
Имя: Иван, Номер: 101, Оценка: 85.50
93          serializeStudents("journal.dat");
(gdb) n
Сериализация завершена.
95          return 0;
(gdb)
```

Листинг:
1lab.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Структура для представления информации о студенте
struct Student {
    char name[50];
    int roll_number;
    float grade;
    struct Student* next;
};

// Глобальная переменная для хранения указателя на начало списка
struct Student* head = NULL;

// Функция для вставки нового студента в начало списка
void insertStudent(char name[], int roll_number, float grade) {
    struct Student* newStudent = (struct Student*)malloc(sizeof(struct Student));
    strncpy(newStudent->name, name, sizeof(newStudent->name));
    newStudent->roll_number = roll_number;
    newStudent->grade = grade;
    newStudent->next = head;
    head = newStudent;
    printf("Студент добавлен в журнал.\n");
}

// Функция для удаления студента по его номеру
void deleteStudent(int roll_number) {
    struct Student* current = head;
    struct Student* prev = NULL;

    while (current != NULL) {
        if (current->roll_number == roll_number) {
            if (prev == NULL) {
                head = current->next;
            } else {
                prev->next = current->next;
            }
            free(current);
            printf("Студент удален из журнала.\n");
            return;
        }
        prev = current;
        current = current->next;
    }

    printf("Студент с номером %d не найден.\n", roll_number);
}
```

```
// Функция для отображения списка студентов
void displayStudents() {
    struct Student* current = head;
    if (current == NULL) {
        printf("Журнал успеваемости пуст.\n");
    } else {
        printf("Журнал успеваемости студентов:\n");
        while (current != NULL) {
            printf("Имя: %s, Номер: %d, Оценка: %.2f\n", current->name, current-
>roll_number, current->grade);
            current = current->next;
        }
    }
}

// Функция для сериализации списка студентов в файл
void serializeStudents(const char* filename) {
    FILE* file = fopen(filename, "wb");
    if (file == NULL) {
        perror("Ошибка при открытии файла для записи");
        return;
    }

    struct Student* current = head;
    while (current != NULL) {
        fwrite(current, sizeof(struct Student), 1, file);
        current = current->next;
    }

    fclose(file);
    printf("Сериализация завершена.\n");
}

int main() {
    insertStudent("Иван", 101, 85.5);
    insertStudent("Мария", 102, 92.0);
    insertStudent("Петр", 103, 78.2);

    displayStudents();

    deleteStudent(102);

    displayStudents();

    serializeStudents("journal.dat");

    return 0;
}
```

Вывод:

В данной лабораторной работе я создал программу на языке программирования C для манипуляции данными на основе рекурсивных структур, представляющую электронный журнал успеваемости студентов. Моя программа включает в себя функции вставки, удаления, навигации и сериализации списка студентов.

Программа включает следующие основные функции:

1. **insertStudent:** Эта функция позволяет вставлять новых студентов в начало списка. Она принимает имя студента, номер студента и его оценку, создает новую структуру студента и добавляет ее в начало списка.
2. **deleteStudent:** Функция удаляет студента из списка по его номеру. Если студент с указанным номером не найден, программа выведет сообщение об ошибке.
3. **displayStudents:** Функция отображает список всех студентов в журнале. Если журнал пуст, программа сообщит об этом.
4. **serializeStudents:** Эта функция выполняет сериализацию списка студентов в бинарный файл. В данном случае, файл назван "journal.dat".

Для отладки программы, я использовали отладчик GDB (GNU Debugger).

Выполнил следующие шаги следовали следующим шагам:

1. Компиляция программы с флагом **-g** для включения информации для отладки: **gcc -g -o app 1lab.c**.
2. Запуск GDB с помощью команды **gdb ./app**.
3. В GDB установил точку останова в функции **main**, чтобы начать трассировку с начала выполнения программы: **break main**.
4. Запустил программу в GDB с помощью команды **run**.
5. Выполнил шаги пошагово с помощью команд **step**, **next**, **continue**, а также просматривал значения переменных с помощью команды **print**.
6. По завершении трассировки, выштk из GDB с помощью команды **quit**.